

Building MCP Servers with Go

Understanding the Protocol Behind AI-Data Interactions

Christian Bargmann

About Me

Christian Bargmann (It's me :D)

Developer Experience Team @ MOIA

Professional Focus:

- Distributed systems & developer productivity
- Observability, reliability, maintainability
- Platform engineering with AWS & Kubernetes
- Open source contributor

Community:

- Co-organizer: CNCF Hamburg CloudNative / Kubernetes meetup
- Active on GitHub: exploring new tools and techniques
- Passionate about practical, impactful technology solutions

Connect:

- GitHub: github.com/cbrgm (<https://github.com/cbrgm>)
- Blog: cbrgm.net (<https://cbrgm.net>)
- LinkedIn: [linkedin.com/in/bargmann](https://www.linkedin.com/in/bargmann/) (<https://www.linkedin.com/in/bargmann/>)

What is MCP?

MCP (Model Context Protocol) standardizes how AI systems connect to data sources and tools

Before MCP

- Custom integrations for each AI application
- Fragmented ecosystem
- Duplicate integration work
- No interoperability

With MCP

- Open, standardized protocol
- Reusable server components
- Universal AI-data connections
- Industry-wide adoption

Key Insight: MCP represents a shift from simple API calls to bidirectional communication channels, enabling AI models to autonomously access context and execute tools.

Open-sourced by Anthropic (November 2024) • Adopted by OpenAI, Google DeepMind, and major toolmakers

Why MCP? Design Principles

■ The Problem MCP Solves

Traditional API Integration:

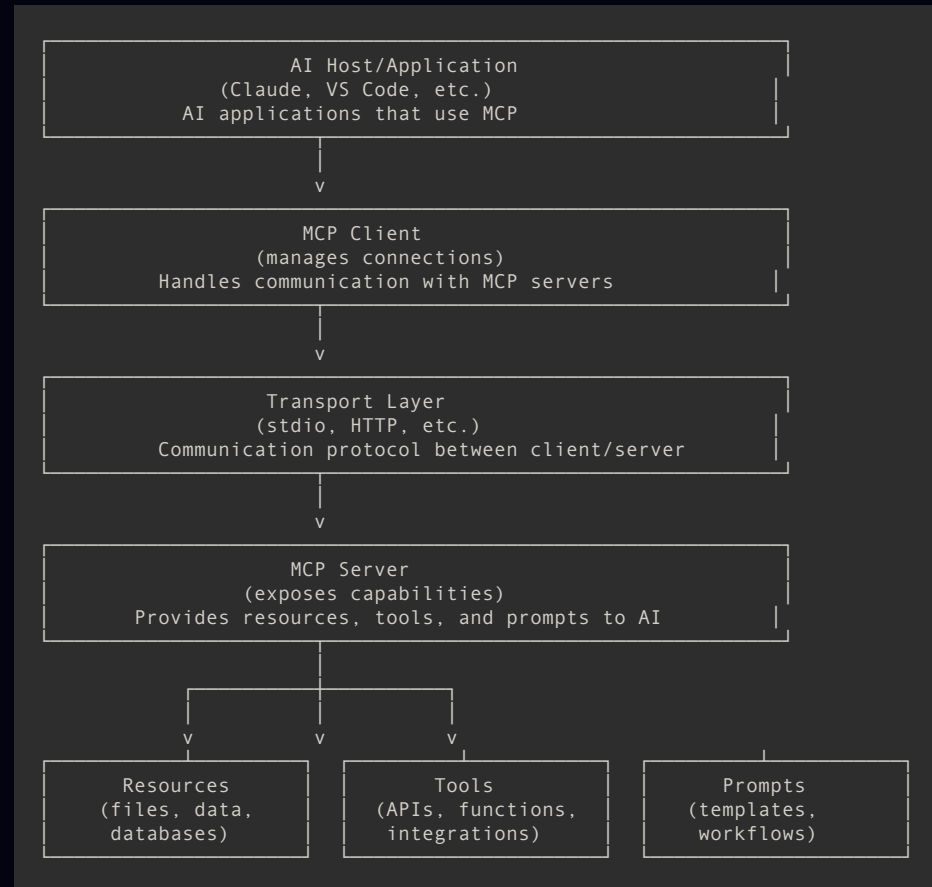
- Each AI app builds custom connectors
- Data flows one direction (request → response)
- AI has no persistent context
- Integration logic tightly coupled to application

MCP's Architectural Shift:

- Bidirectional Communication - AI can discover and explore capabilities
- Standardized Protocol - Write once, use across all AI systems
- Context-Aware - AI maintains connection to data sources
- Separation of Concerns - Data sources expose capabilities, AI decides how to use them

Core Design Decision: Move from "AI calls API" to "AI collaborates with data source"

How MCP Works & Architecture Overview



Transport Mechanisms

How MCP Communicates

Design Principle: Transport abstraction separates protocol from communication method

Stdio Transport

- Local processes
- Command-line tools
- Zero network overhead
- Natural for desktop integration

```
npm @modelcontextprotocol/server-filesystem /path
```

Use Case: Claude Desktop, local tooling

Streamable HTTP Transport

- Stateless, infrastructure-friendly
- Bidirectional over standard POST/GET
- Optional SSE for server-to-client streaming
- Resumable streams with per-stream IDs

```
{  
  "endpoint": "https://api.example.com/mcp",  
  "method": "POST"  
}
```

Use Case: Cloud services, enterprise deployments

Note: MCP evolved from HTTP+SSE to Streamable HTTP for better reliability and infrastructure compatibility.

Protocol Structure

JSON-RPC 2.0 Foundation

Why JSON-RPC?

- Proven protocol with wide language support
- Request/response correlation via id
- Clear error handling semantics
- Human-readable for debugging

Request:

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "create_issue",
    "arguments": {
      "title": "Bug: Login fails"
    }
  },
  "id": 1
}
```

Success Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "content": [{
      "type": "text",
      "text": "Created issue #1234"
    }]
  },
  "id": 1
}
```

Error Response:

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32602,
    "message": "Invalid params"
  },
  "id": 1
}
```

Building MCP Servers

■ Mental Model & Interface

Server Lifecycle:

1. Initialize - Declare capabilities
2. Discovery - Client explores what you offer
3. Execute - Handle requests
4. Notify - Push updates (optional)

Key Methods to Implement:

- `initialize` / `initialized` - Handshake
- `resources/list` / `resources/read`
- `tools/list` / `tools/call`
- `prompts/list` / `prompts/get`

Mental Model: Think of your server as a capability provider, not a traditional API.

- Stateful Connection - Maintains session
- Lazy Loading - Discovered on demand
- AI-Driven - Client decides flow
- Your Job - Validate, execute safely, return structured results

Key Insight: The AI explores and orchestrates - you provide the building blocks.

How LLMs Discover MCP Capabilities

The Discovery & Injection Flow

The Flow:

1. MCP Client connects to server
2. Server returns capabilities:
 - Tool schemas with descriptions
 - Resource URIs and metadata
 - Prompt templates
3. Client injects these into LLM context
4. LLM sees available tools as part of its prompt
5. LLM decides when to call tools based on user request
6. Client executes via MCP protocol

Key Separation:

- MCP Client - Understands JSON-RPC protocol
- Server - Provides clear descriptions
- LLM - Sees tool descriptions in context, makes decisions

Crucial Point: The LLM doesn't "know" MCP protocol. It sees:

```
Available tools:  
- update_inventory(tea_name, quantity, operation)  
  "Update tea inventory levels"
```

Then decides: "User wants to update Earl Grey stock, I'll call update_inventory."

Design Principle: Self-describing capabilities enable autonomous AI behavior.

Core Components

Resources, Tools, and Prompts

Resources Read-only data

- URI-based addressing
- Text or binary content
- Application-controlled

Examples:

- `tea://inventory/tea`
- `file:///docs/api.md`
- `db://customers/1234`

Tools AI-controlled actions

- Defined input schemas
- Execute operations
- Return structured results

Examples:

- `update_inventory`
- `create_github_issue`
- `query_database`

Prompts User-controlled workflows

- Template with arguments
- Combine resources + tools
- Multi-step guidance

Examples:

- `analyze_codebase`
- `generate_report`
- `debug_workflow`

How They Work Together: Resources provide context → Tools enable actions → Prompts orchestrate workflows

MCP Server Examples

■ Production-Ready MCP Servers

Official Servers:

- Filesystem - Secure file operations with path restrictions
- Git - Repository management and version control
- GitHub - Issues, PRs, repository access
- Memory - Persistent context using knowledge graphs
- Postgres - Database queries and schema introspection

Community Servers:

- Slack - Team communication and channel management
- Google Drive - Document access and collaboration
- Puppeteer - Browser automation and web scraping

Implementing with Go

■ Why Go + Available SDKs

Go's Natural Fit:

- Goroutines - Handle multiple connections
- Channels - Bidirectional communication
- Context - Request lifecycle management
- Single Binary - Easy deployment

MCP's event-driven model maps naturally to Go's concurrency primitives.

Available SDKs:

- Official Go SDK (<https://github.com/modelcontextprotocol/go-sdk>)
 - Maintained with Google
- mcp-go (<https://github.com/mark3labs/mcp-go>) - Community implementation
- mcp-golang (<https://github.com/metoro-io/mcp-golang>) - Alternative library

Bonus: Generate servers from OpenAPI specs or Protocol Buffers (protoc-gen-go-mcp)

Client Configuration

Connecting Your Server

Claude Desktop (`claude_desktop_config.json`):

```
{
  "mcpServers": {
    "teashop": {
      "command": "./teashop-mcp-server"
    }
  }
}
```

VS Code (settings):

```
{
  "mcp.servers": {
    "teashop": {
      "command": "go",
      "args": ["run", "main.go"]
    }
  }
}
```

Key Point: Stdio transport makes integration simple - just specify the command and args.

Security Considerations

■ Key Points for Production

Security Essentials:

- Input Validation - Validate and sanitize all parameters
- Access Control - Implement authorization at transport layer
- Path Traversal - Sanitize file paths and URIs
- Rate Limiting - Prevent resource exhaustion
- Prompt Injection - Be aware of LLM-specific attack vectors

Design Principle: No built-in authentication - implement at your chosen transport layer (HTTP middleware, process isolation, etc.)

Performance Characteristics:

- Lightweight JSON-RPC protocol
- Stateful connections reduce handshake overhead
- Pagination recommended for large datasets
- Go's concurrency handles multiple clients efficiently

Getting Started

Implementation Roadmap

1. Design Your Server:

- Identify what data to expose (Resources)
- Define what actions AI can take (Tools)
- Create workflow templates (Prompts)

2. Implement with Go SDK:

```
server := mcp.NewServer(&mcp.Implementation{Name: "myserver"}, nil)
mcp.AddTool(server, &mcp.Tool{...}, handler)
server.Run(ctx, &mcp.StdioTransport{})
```

3. Test:

- MCP Inspector - `npm install -g @modelcontextprotocol/inspector`
- Manual JSON-RPC via stdio
- Claude Desktop for real usage

Resources:

- modelcontextprotocol.io (<https://modelcontextprotocol.io>) - Specification
- github.com/modelcontextprotocol/go-sdk (<https://github.com/modelcontextprotocol/go-sdk>) - Official SDK

Key Takeaways

■ Core Concepts

Architectural Shift:

- Bidirectional Communication - Beyond request/response to persistent connections
- Self-Describing - Capabilities discovered at runtime, injected into LLM context
- Three Primitives - Resources (data), Tools (actions), Prompts (workflows)
- Transport Abstraction - Same protocol works over stdio, Streamable HTTP, or custom transports

Why Go Works Well:

- Event-driven protocol maps to goroutines and channels
- Context package aligns with MCP's request lifecycle
- Single binary simplifies deployment

Design for Autonomy: Clear descriptions and well-defined schemas enable AI to discover, understand, and use your capabilities without human intervention.

Thank You!

■ Questions & Discussion

Building MCP Servers with Go Understanding the Protocol Behind AI-Data Interactions

Christian Bargmann

- GitHub: github.com/cbrgm (<https://github.com/cbrgm>)
- Blog: cbrgm.net (<https://cbrgm.net>)
- LinkedIn: [linkedin.com/in/bargmann](https://www.linkedin.com/in/bargmann/) (<https://www.linkedin.com/in/bargmann/>)

DevFest Hamburg 2025

